

87. Quantifiers: `+`, `*`, `?`, `{n}`

Maano string `+7(903)-123-45-67` me saare **numbers** dhundhne hain (`7`, `903`, `123`, `45`, `67`), sirf single digits nahi. Number = 1 ya zyada digits (`\d`). Kitne chahiye ye batane ke liye **quantifier** lagate hain.

Quantity `{n}`

Curly braces me number. Quantifier kisi character (ya character class, ya `[...]` set) ke baad lagta hai.

- **Exact count** `{5}`: `\d{5}` = bilkul 5 digits (`\d\d\d\d\d`).

```
js
"I'm 12345 years old".match(/\d{5}/);    // "12345"
```

Lambe numbers ko hatane ke liye `\b\d{5}\b`.

- **Range** `{3,5}`: 3 se 5 baar.

```
js
"I'm not 12, but 1234 years old".match(/\d{3,5}/);    // "1234"
```

- **Upper limit hata do** `{3,}`: 3 ya zyada.

```
js
"I'm not 12, but 345678 years old".match(/\d{3,}/);    // "345678"
```

Numbers ke liye `\d{1,}`:

```
js
"+7(903)-123-45-67".match(/\d{1,}/g);    // 7,903,123,45,67
```

Shorthands

- `+`: "ek ya zyada", `{1,}` jaisa. `\d+` numbers dhundhta hai.
- `?`: "zero ya ek", `{0,1}` jaisa (symbol ko **optional** banata hai). `colou?r` = `color` aur `colour` dono.

```
js
```

```
"Should I write color or colour?".match(/colou?r/g); // color, colour
```

- `(*)`: "zero ya zyada", `{0,}` jaisa. `(\d0*)` = ek digit ke baad kitne bhi (ya koi nahi) zero.

js

```
"100 10 1".match(/\d0*/g); // 100, 10, 1  
"100 10 1".match(/\d0+/g); // 100, 10 (1 nahi, '0+' ko kam se kam ek zero c
```

Aur examples

Quantifiers complex regexps ke main "building blocks" hain.

Decimal fractions: `(\d+\.\d+)`

js

```
"0 1 12.345 7890".match(/\d+\.\d+/g); // 12.345
```

Opening HTML tag (bina attributes ke):

1. Sabse simple: `(/<[a-z]+>/i)` -> `<body>`
2. Behtar: `(/<[a-z][a-z0-9]*>/i)` (standard ke hisaab se tag name me digit pehle ke alawa kahin bhi ho sakta hai, jaise `<h1>`)

Opening ya closing tag: `(/<\/?[a-z][a-z0-9]*>/i)`. Shuruaat me optional slash `(/?)` joda, jise backslash se escape karna padta hai, warna JS use pattern ka end samjhega.

js

```
"<h1>Hi!</h1>".match(/<\/?[a-z][a-z0-9]*>/gi); // <h1>, </h1>
```

Regexp ko zyada precise banana ho to wo zyada lamba aur complex hota hai. Jaise HTML tags ke liye simple `<\w+>` bhi chal sakta hai, par HTML me tag name ki sakht restrictions hain, to `<[a-z][a-z0-9]*>` zyada reliable hai. Real life me dono acceptable, ye is par depend karta hai ki "extra" matches ko kitna sahen aur unhe alag se hatana kitna aasan hai.

Tasks

- Ellipsis "... (3 ya zyada dots): `(/\.{3,}/g)` (dot special hai, isliye `(\.)`).
- HTML colors `(\#[a-f0-9]{6}\b/gi)`: `(\b)` end me isliye ki `(#12345678)` (8 hex) me `(#123456)` na mile.